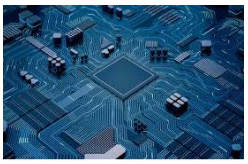
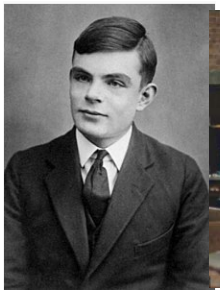
NSI S10 : Architecture matérielle

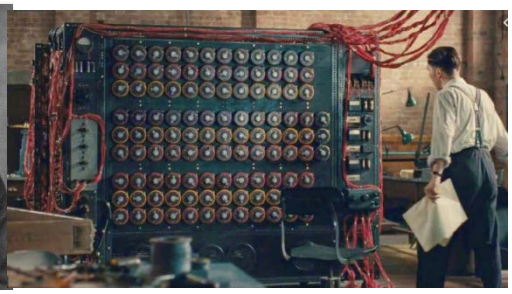


1. Architecture des ordinateurs :

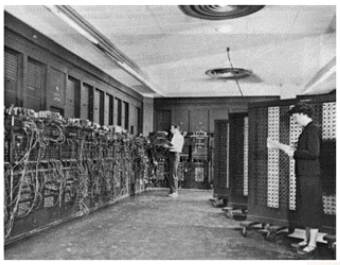
a) Evolution architecturale :

1936 : Machine de Turing
Modèle abstrait du fonctionnement des appareils mécaniques de calcul, tel un ordinateur créé par Alan Turing.



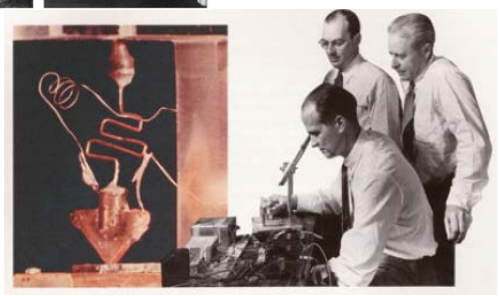


1946 : Ordinateur ENIAC
1er ordinateur électronique universel
30m, 30 tonnes, 18000 tubes à vide
Mauchly et Eckert (à droite)

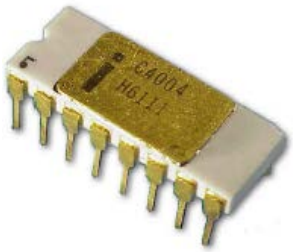




1947 : Invention du transistor
Avancée prodigieuse de l'électronique en coût, taille, fiabilité, consommation
Shockley (assis), Bardeen (à gauche) et Brattain (à droite), 16/12/1947

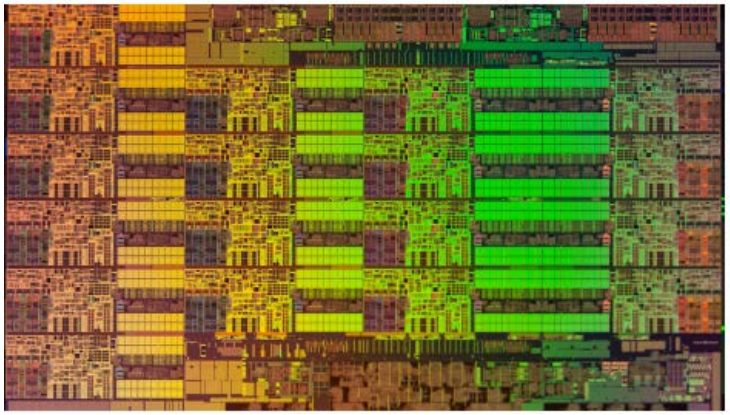


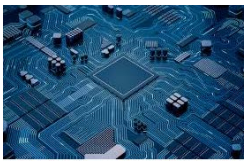
1971 : Intel 4004
1er microprocesseur commercial
2300 transistors dans un unique circuit intégré
Fréquence de 740 kHz, 0,092 MIPS
Hoff, Faggin, Mazor et Shima (de haut en bas et de gauche à droite)





2016 : Intel Broadwell
3.1 GHz
Environ 4000 MIPS.
7,2 milliards de transistors
18 cœurs.
Toujours architecture « von Neumann »

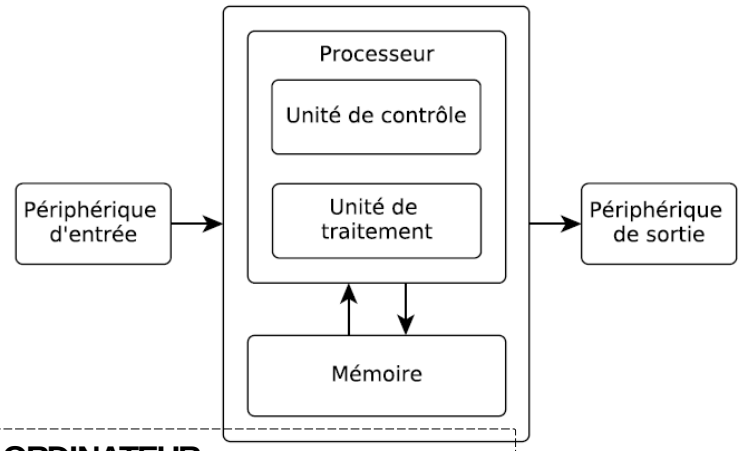


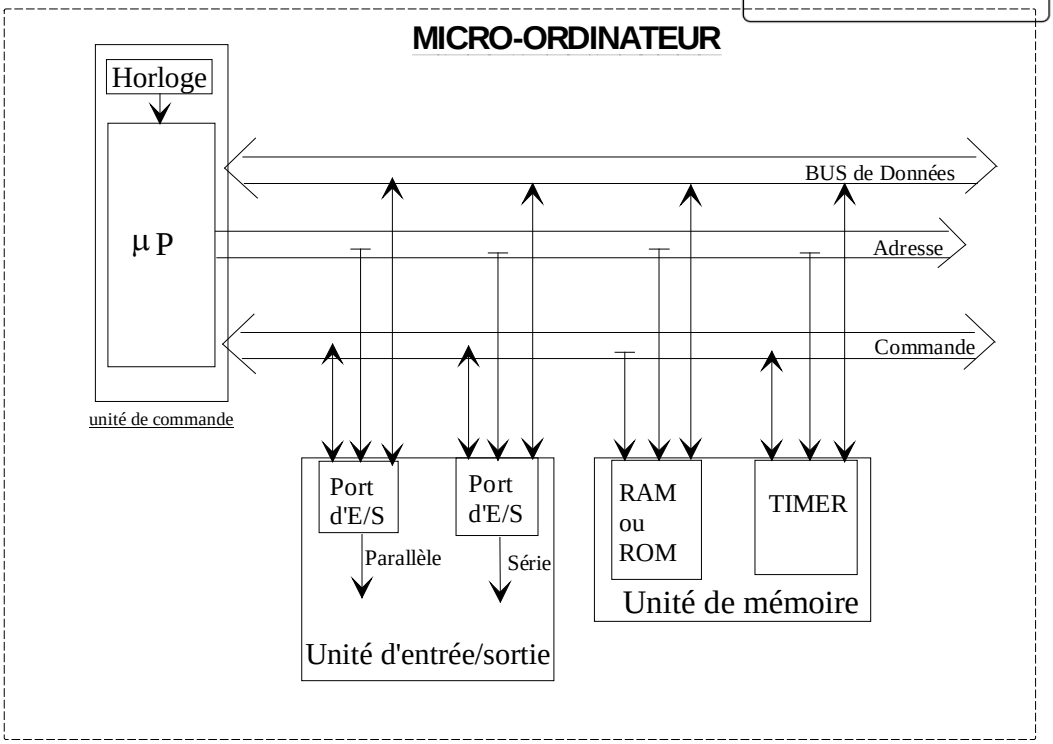


b) Base de l'architecture des processeurs :

Quasiment tous les ordinateurs modernes reposent sur l'architecture « à programmes enregistrés » dite « de **von Neumann** ».

- Instructions et données stockées dans une même mémoire contrairement à l'architecture de certain microcontrôleur (architecture de type Harvard).
- Cellules mémoires identifiables par leur adresse.
- Traitement (processeur) et stockage (mémoire) séparés.
- Instructions exécutées séquentiellement.





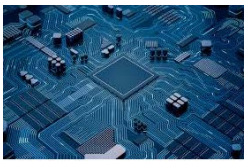
Il est fondamental de faire la distinction entre un ordinateur et un microprocesseur. Un micro-ordinateur est constitué de plusieurs éléments dont le plus important est le microprocesseur. Ce dernier est généralement une puce intégrée renfermant tous les circuits de l'unité de commande et de l'unité arithmétique et logique.

Bus de données : C'est là où circulent les données. (bidirectionnel)
Adresse: permet au µP de sélectionner où va (d'où vient) une donnée
Commande: Horloge, interruptions, lecture, écriture.

Fonctions réalisées par un µP

Un µP peut :

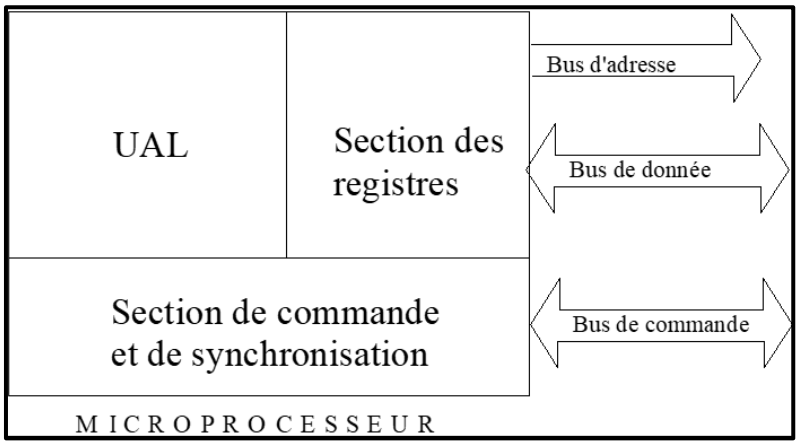
- *Fournir* les signaux de synchronisation et de commande à tous les éléments du micro-ordinateur.
- *Prendre en charge les instructions* et les données dans la mémoire
- *Transférer* les données entre la mémoire et les dispositifs E/S et vice versa
- *Décoder* les instructions
- *Effectuer les opérations arithmétiques et logiques* commandées par les instructions
- Réagir aux signaux de commande produits par les entrées sorties comme le signal RESET et les INTERRUPTIONS.



c) Organisation interne d'un microprocesseur

Le microprocesseur renferme tous les circuits logiques nécessaires à la réalisation des fonctions précédentes. La logique interne est d'une très grande complexité on peut cependant schématiser et dire qu'elle comprend 3 sections comme le montre la figure ci-dessous.

- L'UAL, pour sa part a pour tâche d'effectuer sur les données les opérations arithmétiques et logiques (ET, OU, décalages, incrémentation, décrémentation etc.…).
- La Section de registres contient les différents registres qui ont tous un rôle précis. Le plus important des registres est le compteur ordinal ou pointeur d'instruction PI (instruction pointer en anglais). Il contient l'adresse de la **prochaine instruction** à exécuter.
- Section de commande et de synchronisation : son rôle est de prendre en charge (récupérer) et décoder (interpréter) les codes instruction dans la mémoire contenant le programme (R/W,horloge,)



i. Execution d'une instruction

- Chargement de l'instruction à exécuter
- Décodage de l'instruction
- Incrémentation du compteur ordinal
- Localiser en mémoire les données nécessaires.
- Charger si nécessaire les données dans l'UAL
- Exécuter l'instruction puis recommencer

ii. Mots machines

La plus petite unité d'information dans un ordinateur est le bit.
Un bit isolé ne nous apprend pas grand-chose. C'est pour cela que l'on a choisi comme unité d'information dans un ordinateur **un groupe de bits appelé mot**. Le nombre de bits qui constitue un mot est le paramètre le plus souvent utilisé pour décrire un ordinateur. (8bits, 16bits, 32bits, 64bit).
La longueur du mot sert aussi à indiquer la largeur du bus de données.
Un groupe de 8 bits s'appelle un octet.

Types de mots machines.

Un mot conservé dans une mémoire d'ordinateur peut correspondre à **deux types d'informations** : Une **instruction** ou à **une donnée**.

❖ ***Les données***

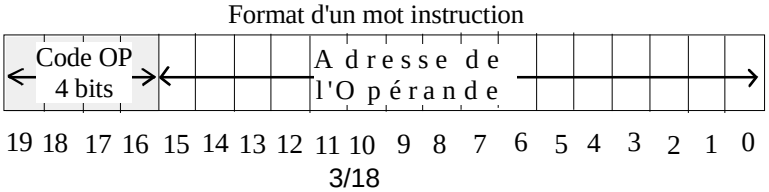
Les données sont des nombres ou des caractères que le programme de l'UCT (unité centrale de traitement) soumet à diverses opérations. Les données se présentent sous diverses formes : binaires signées, DCB (décimal codé en binaire), à virgule flottante, code ASCII.

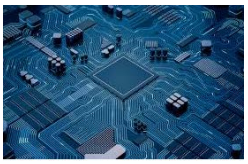
Exemple du nombre +86 dans un mot de 8 bits 01010110
Code ASCII de la lettre V 01010110
Les deux codes sont identiques, l'ordinateur ne sait pas faire la différence entre les deux. C'est la tâche du programmeur de savoir quel type de donnée est mémorisé.

❖ ***Mot d'instruction.***

Le format pour les données varie peu entre ordinateur différent c'est tout le contraire pour les mots instructions. Ces mots contiennent tout ce qui est nécessaire à un ordinateur pour réaliser ses diverses opérations. Le format et les codes employés varient grandement d'un ordinateur à l'autre.
Malgré tout, les mots instruction précisent deux éléments d'information de base :
L'opération à effectuer et l'adresse de l'opérande (donnée) qui est l'objet de l'opération.
Exemple de mot instruction :

Format d'un mot instruction





En utilisant 4 bits pour le code opération, on conclue qu'il y a 16 codes op possibles. En réalité, un ordinateur possède un plus grand nombre d'instructions et doit de ce fait réserver plus de bits pour son code opération.
Le mot d'instruction a réservé 16 bits pour le code adresse de l'opérande. Avec 16 bits on peut adresser 65536 adresses. **En conclusion le mot ci-dessus peut décrire 16 opérations différentes et retrouver 65536 adresses d'opérande.**

d) Exécution d'un programme en langage machine

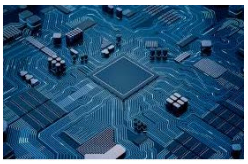
Les mots d'instruction que nous venons de décrire ont reçu le nom d'instructions du langage machine, parce qu'on n'y trouve que des 0 et 1.
Les 2 seuls états que l'ordinateur peut comprendre. De nombreux autres langages peuvent servir à la programmation d'un ordinateur. Les langages évolués ont été conçus pour faciliter le processus de rédaction d'un programme.

Pour illustrer le déroulement d'un programme machine dans un ordinateur nous utiliserons les instructions répertoriées dans le tableau suivant. Ces instructions n'appartiennent à aucun ordinateur en particulier, mais elles sont représentatives des instructions que l'on peut retrouver dans la plupart des micro-ordinateurs.
Chaque instruction est repérée par un mnémonique, c'est à dire l'abréviation qui nous rappelle le code opération.

Exemple de programme :

Adresse Mémoire (HEX)	Mot en mémoire (HEX)	Mnémonique	Description
0000	5B	LDA$0300	Charge la donnée (10) dans l'accumulateur
0001	00		
0002	03		
0003	A2	SUB$0301	soustrait la donnée 5 de l'accumulateur
0004	01		
0005	03		
0006	8A	STA$0302	Mémorise la différence (B) à l'adresse 0302
0007	02		
0008	03		
0009	76	JPZ$0400	si la différence est nulle se branche sur l'adresse 400 où se trouve l'instruction suivante.
000A	00		
000B	04		
000C	3F	HLT	Si la différence est non nulle le programme s'arrête
.	.	.	
.	.	.	
.	.	.	
0300	10		
0301	5		
0302	B		

Mnémonique	Code op Binaire /Hex	Description du fonctionnement
LDA	01011011 /5B	*Charge l'accumulateur*: La donnée mémorisée dans l'adresse de l'opérande est chargée dans l'accumulateur
ADD	01101101 /6D	*Additionne*: L'opérande est additionné au nombre contenu dans l'accumulateur et la somme devient le nouveau nombre dans l'accumulateur
SUB	10100010 /A2	*Soustrait*: l'opérande est retranché du contenu de l'accumulateur et la différence devient le nouveau nombre dans l'accumulateur.
STA	10001010 /8A	*Charge l'accumulateur*: Le contenu de l'accumulateur est chargé en mémoire à l'adresse spécifiée par l'opérande.
JMP	01001100 /4C	*Branchement (non conditionnel)* l'instruction suivante est trouvée à l'endroit indiqué par l'adresse de l'opérande, et non dans une case mémoire qui suit
JPZ	01110110 /76	*Branchement si zéro*: L'instruction suivant est trouvée dans l'adresse de l'opérande si le contenu de l'accumulateur est zéro. Autrement, elle est dans la case mémoire qui suit.
HLT	00111111 /3F	*Arrêt*: L'ordinateur cesse son travail. Aucune autre instruction n'est plus traitée.



1) L'unité de commande prend en charge le premier octet de l'adresse 0 selon l'indication du PI(**pointeur instruction**).
Cet octet vaut 5B soit le code op de la première instruction.
Les circuits de l'unité de commande évaluent que ce code op particulier signifie LDA (charge l'accumulateur) et que l'adresse de l'opérande doit suivre ce code

2) L'unité de commande incrémente le PI à 0001 et prend en charge l'octet conservé dans la case de cette nouvelle adresse l'octet récupéré est zéro soit l'octet de rang inférieur de l'adresse de l'opérande. L'unité de command incrémente le PI à 0002 et prend en charge l'octet de la case mémoire référencée par cette nouvelle adresse. Cet octet vaut 03 et c'est l'octet de rang supérieur (MSB)

3) L'unité de commande dispose du code op et de l'adresse de l'opérande exécute l'instruction en lisant le mot de l'adresse 300 et en plaçant ce dernier dans l'accumulateur. Ceci termine le cycle d'exécution. (PI=0003)

On remarque un cycle de recherche instruction et un cycle d'exécution de l'instruction. Le PI pointe toujours sur la prochaine (instruction/donnée) à rechercher.

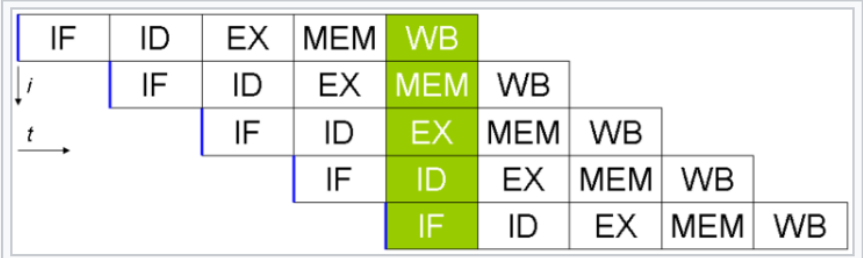
e) PIPE LINE dans un processeur

1. IF (Instruction Fetch) charge l'instruction à exécuter dans le pipeline.
2. ID (Instruction Decode) décode l'instruction et adresse les registres.
3. EX (Execute) exécute l'instruction (par la ou les unités arithmétiques et logiques).
4. MEM (Memory), dénote un transfert depuis un registre vers la mémoire dans le cas d'une instruction du type STORE (accès en écriture) et de la mémoire vers un registre dans le cas d'un LOAD (accès en lecture).
5. WB (Write Back) stocke le résultat dans un registre. La source de ce résultat peut être la mémoire (suite à une instruction LOAD), l'unité de calcul (suite à un calcul à l'étape EX) ou bien un registre (cas d'une instruction MOV).



Séquençage des instructions dans un processeur sans pipeline. Il faut 15 cycles pour exécuter 3 instructions.

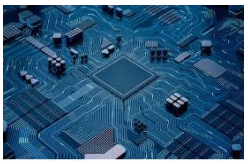
En utilisant la technique du pipeline, notre processeur peut alors contenir plusieurs instructions, chacune à une étape différente.



Séquençage des instructions dans un processeur doté d'un pipeline à 5 étages. Il faut 9 cycles pour exécuter 5 instructions. À t = 5, tous les étages du pipeline sont sollicités, et les 5 opérations ont lieu en même temps.

Remarque :

Un processeur compte de nos jours plusieurs cœurs, on parle couramment de dual-core, quad-core ou d'octo-core parfois. Ainsi, ces processeurs se composent respectivement de deux, quatre ou huit cœurs. Ils sont très importants, car ils agissent comme des « moteurs » indépendants. Ceux-ci permettent de



lancer une ou plusieurs applications de manière simultanée (multitâche) et permettent l'utilisation d'applications lourdes comme des jeux.

f) LES SOCs

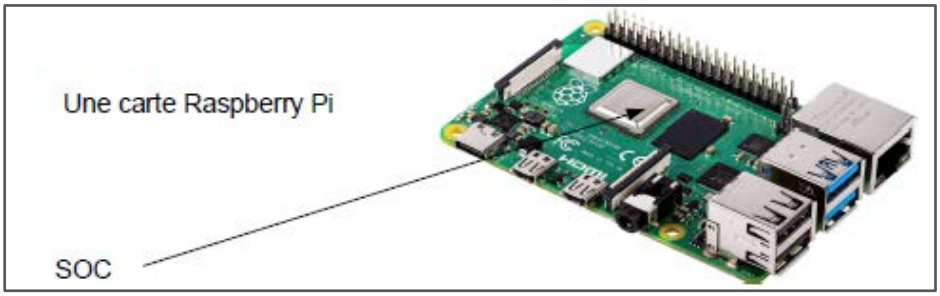
La miniaturisation des circuits électroniques permet de rassembler aujourd'hui sur une même puce, une grande variété d'éléments (microprocesseur, mémoire, interfaces d'entrées-sorties, dispositifs de communication sans fil, capteurs etc …)
Ces systèmes complets, que l'on trouve dans tous les systèmes embarqués, portent le nom de systèmes sur puce (Systems on chip, d'où l'acronyme SOCs)

- Des exemples :

Certains appareils photo numériques intègrent des SOCs de traitement d'image. Ces fonctions devaient être auparavant remplies par des applications logicielles.

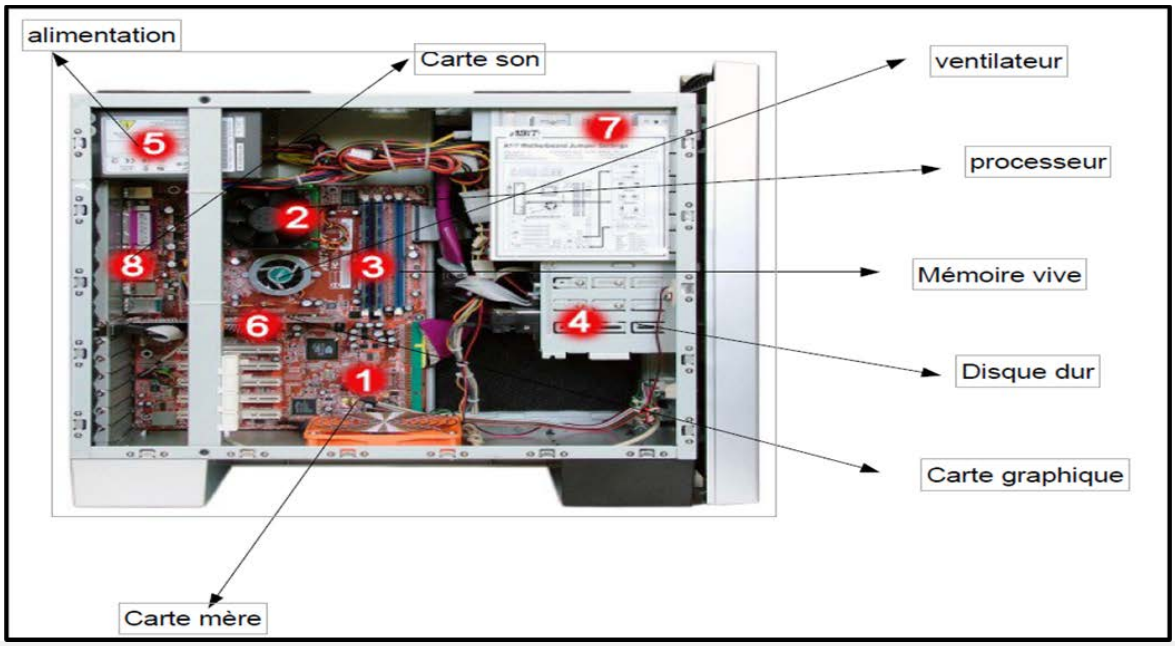
Il existe des ordinateurs miniaturisés complets, dits nano ordinateurs, basés sur l'utilisation de SOCs très complets, voire sur un seul SOC qui regroupe toutes les fonctionnalités d'un ordinateur classique. Par exemple les Arduinos et les Raspberry Pi intègrent un SOC.

Le SoC s'est popularisé avec l'informatique nomade, en particulier les smartphones et les tablettes tactiles ainsi que les systèmes embarqués

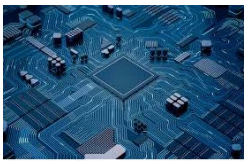


Avantages	**Inconvénient**
Faible consommation Faible encombrement	Aucune mise à jour matériel possible

g) Ordinateur traditionnel

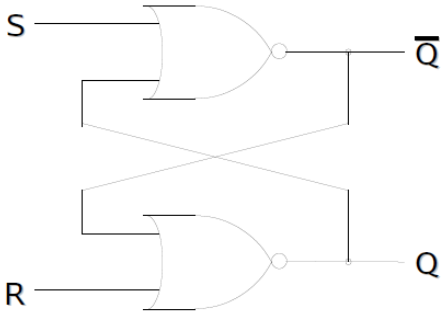


Avantages	**Inconvénient**
Mise à jour matériel possible	Encombrement Consommation



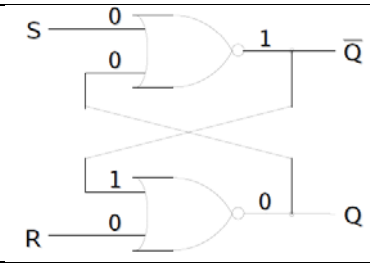
2. Logique séquentielle :

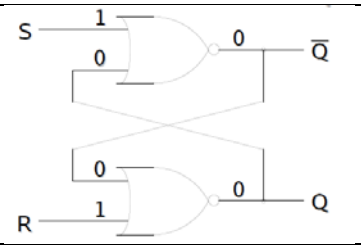
Pour stocker les informations, il faut un circuit capable de « se souvenir » de la dernière valeur d'entrée qui lui a été fournie
On peut construire un tel circuit à partir de deux portes NAND ou deux portes NOR rebouclées



a) LA BASCULE RS

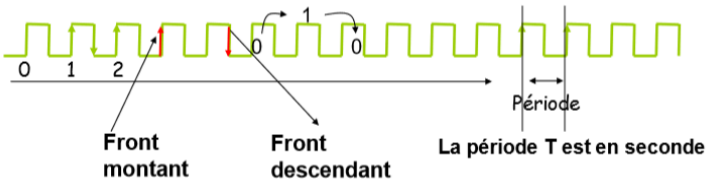
- Une bascule RS est une fonction qui a deux entrées et deux sorties.
- Les deux sorties Q et $\bar{Q}$ sont complémentaires l'une de l'autre.

Si S et R valent 0, et que Q vaut 0, alors : • $\bar{Q} = 1$ • Cette configuration est stable	

• Lorsque S et R sont simultanément à 1, on est dans un état stable dans lequel **Q et $\bar{Q}$ valent 0** • Lorsque S et R repassent simultanément à 0, l'état final de la bascule est non prévisible	

b) Horloge (Clock)

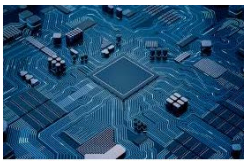
Horloge : composant passant indéfiniment et régulièrement d'un niveau haut à un niveau bas (succession de 1 et de 0), chaque transition s'appelle un top.

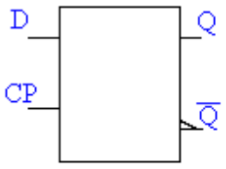


Fréquence = nombre de changement par seconde en hertz (Hz)
Fréquence = 1/période
Une horloge de **1 hertz** a une période de **1 seconde**.
Une horloge de **1 megahertz** a une période de **1 microseconde**.
Une horloge de **1 gigahertz** a une période de **1 nanoseconde**.

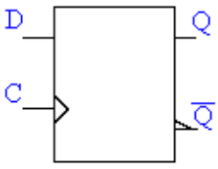
c) La bascule D

Les bascules sont des éléments bistables qui peuvent prendre l'état O ou l'état 1, et le conserver ; en cela elles constituent ***une mémoire élémentaire***.





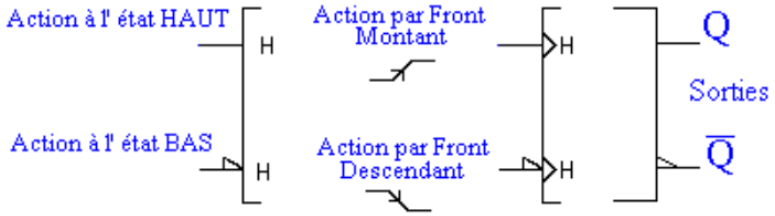
Bascule D active par niveau



Bascule D a commande sur front

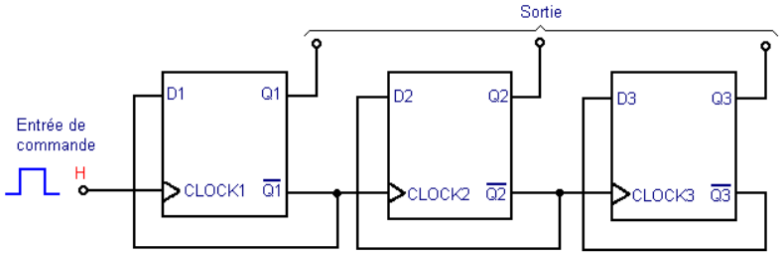
Table de vérité

D	C	Q_n
X	0	Q_{n-1}
0	1	0
1	1	1

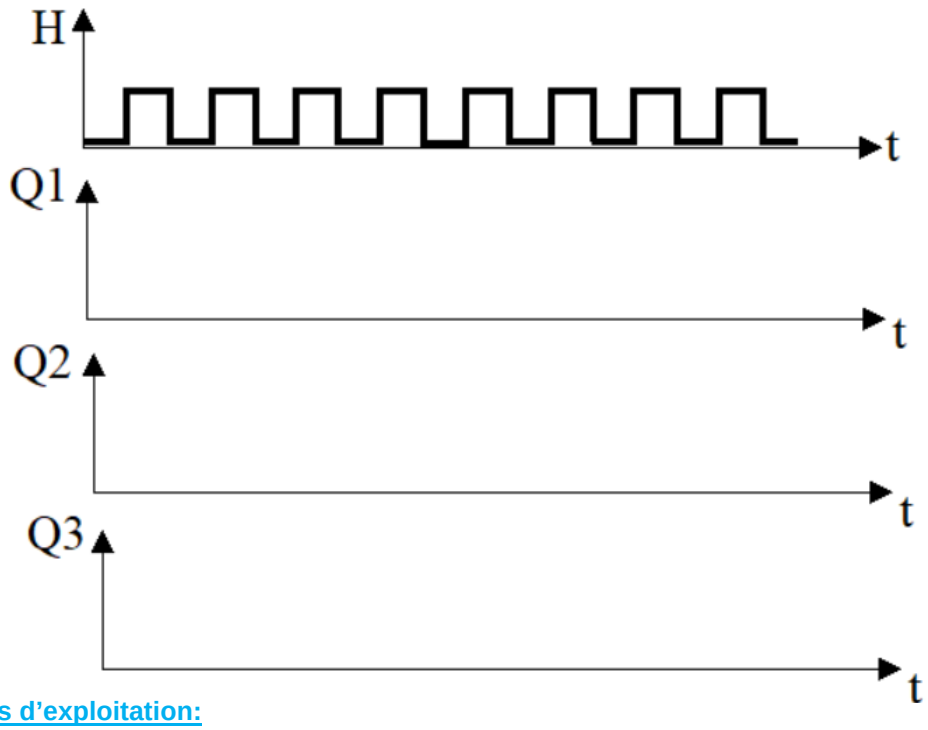


Applications des circuits séquentiels : les registres , les compteurs, les mémoires

Soit le montage suivant :



1) Compléter le chronogramme suivant. Que fait ce montage ?



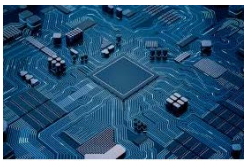
3. Les systèmes d'exploitation:

a) Introduction :

La genèse des systèmes d'exploitation (Windows, mac OS, android, Linux et unix) qui ont été inventés au milieu des années 1970 : https://youtu.be/4OhUDAtmAUo

Un système d'exploitation est un intermédiaire entre le matériel physique et les applications utilisées sur le système informatique (ordinateur, tablette, téléphone portable…). Il s'occupe de :

- la gestion des fichiers,



- la gestion de la mémoire,
- la gestion des applications (processus en cours d'exécution)
- la gestion des entrées/sorties.

Histoire d'UNIX à l'origine des systèmes d'exploitation modernes né en 1972 :
https://youtu.be/Za6vGTLp-wg



Tux, la mascotte de Linux

Le système UNIX est un système dit "propriétaire", c'est-à-dire un système non libre.
En 1991, un étudiant finlandais, Linus Torvalds, décide de créer un clone libre d'UNIX en ne partant de rien puisque le code source d'UNIX n'est pas public. Ce clone d'UNIX va s'appeler Linux (Linus+UNIX) :
https://youtu.be/IquNF_DXcF8



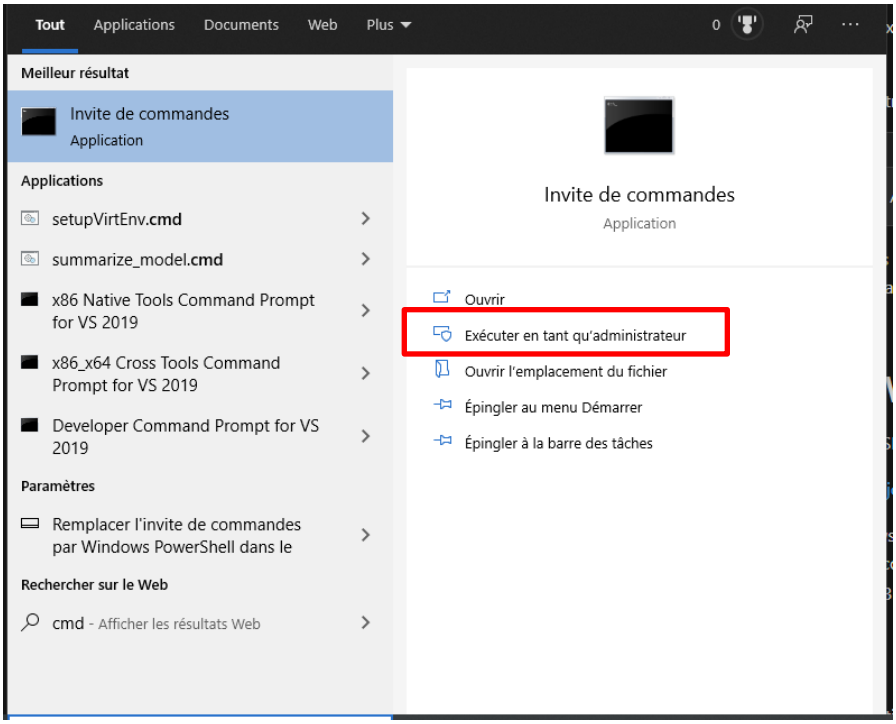
Bill Gates et Paul Allen

Microsoft a été créée par Bill Gates et Paul Allen en 1975. Microsoft est surtout connue pour son système d'exploitation Windows. Windows est un système d'exploitation "propriétaire", la première version de Windows date 1983, mais à cette date Windows n'est qu'un ajout sur un autre système d'exploitation nommé MS-DOS.

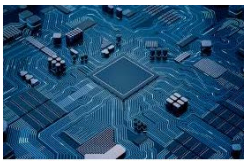
Le système d'exploitation des ordinateurs de marque Apple : tous les ordinateurs d'Apple sont livrés avec le système d'exploitation macOS. Ce système macOS est un système d'exploitation UNIX, c'est donc un système d'exploitation propriétaire.
Pour faire la suite des exercices, nous allons installer une machine virtuelle avec comme système d'exploitation Ubuntu (distribution linux donc libre de droit).

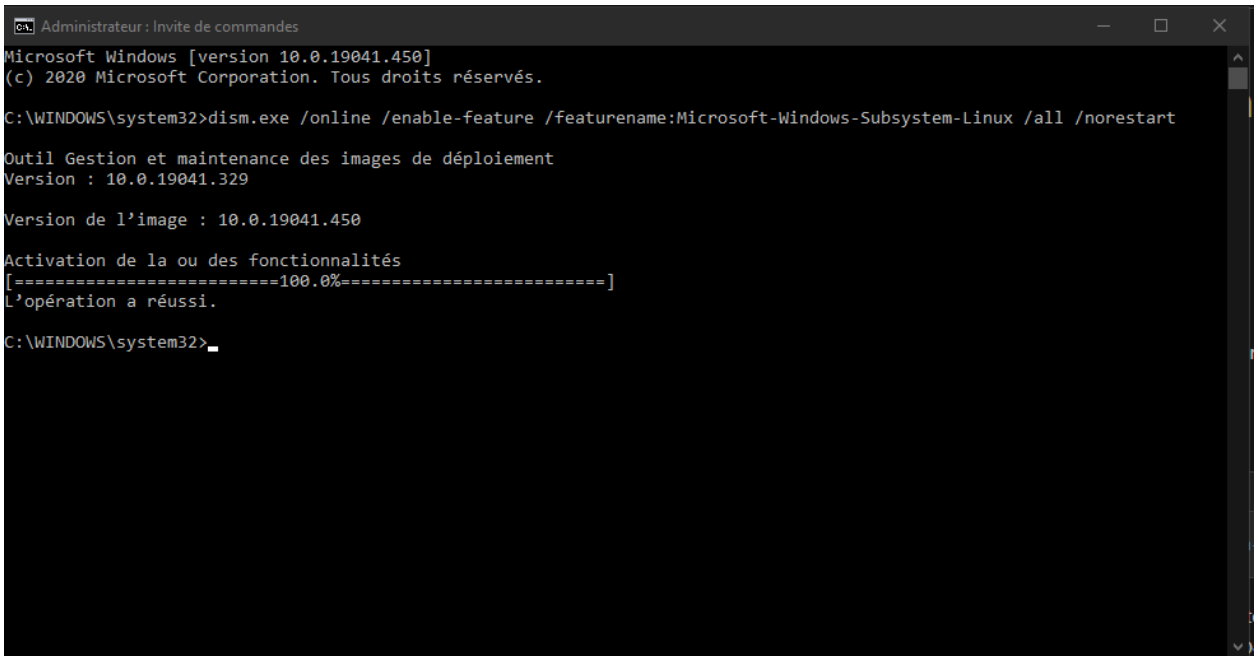
b) Installation :

2) Dans la barre de recherche taper Cmd puis exécuter en tant qu'administrateur.



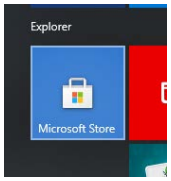
3) Taper :
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart



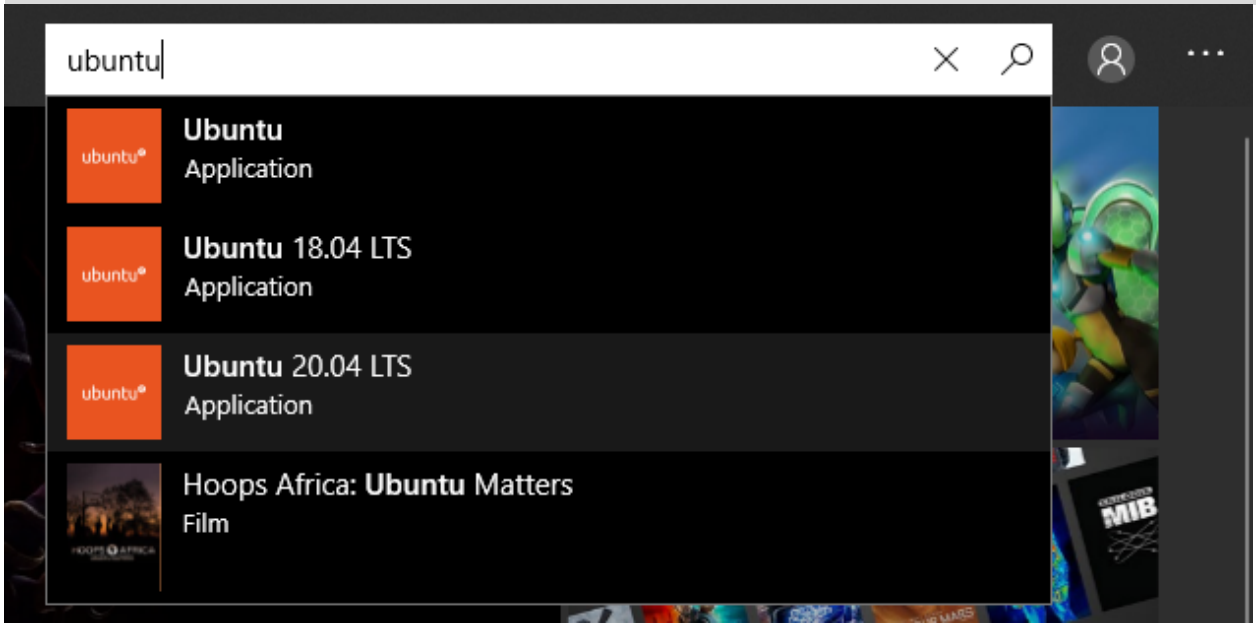


4) Redémarrer votre pc

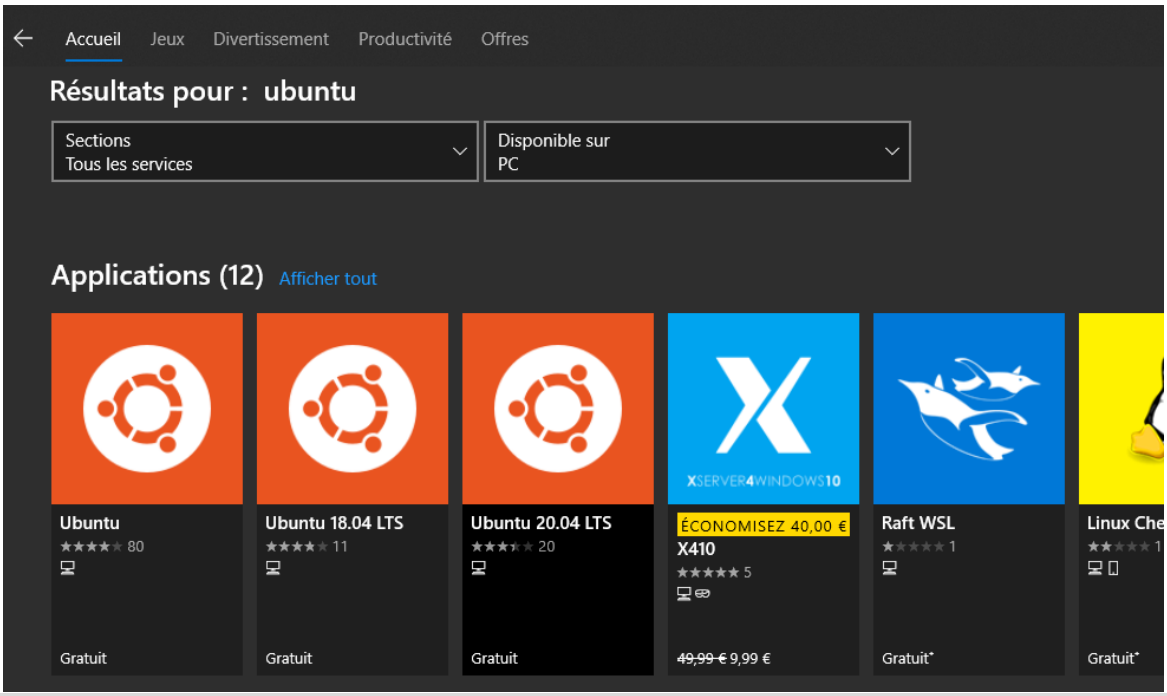
5) Ouvrir Microsoft store :



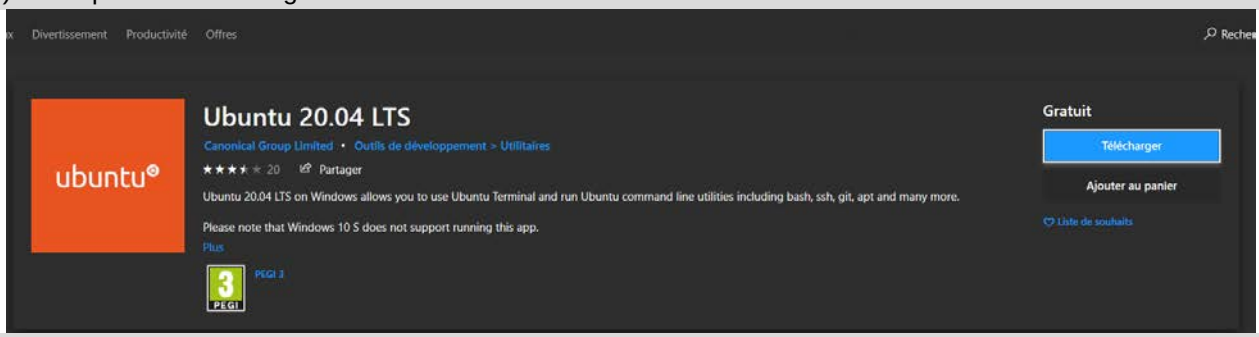
6) Rechercher l'application « Ubuntu »



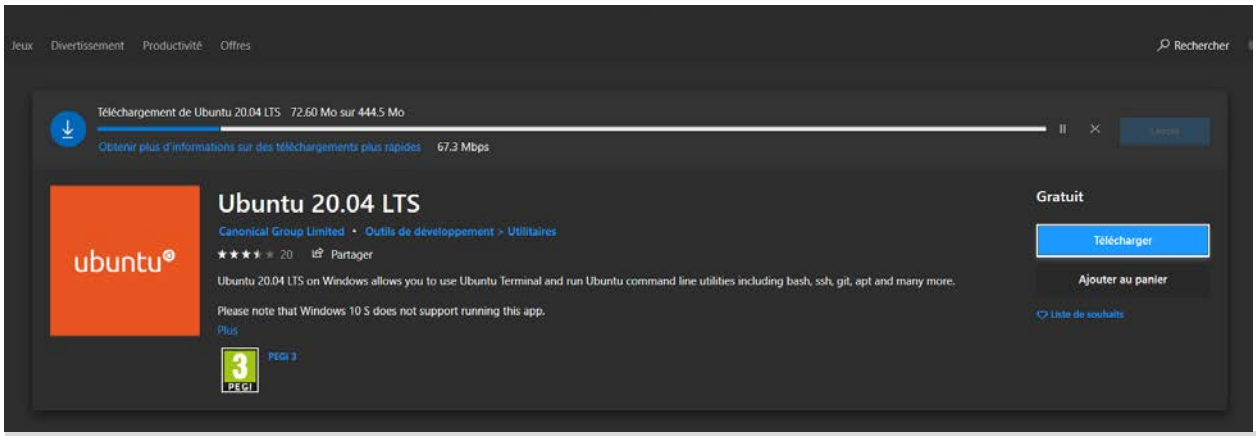
7) Choisir Ubuntu 20.04LTS



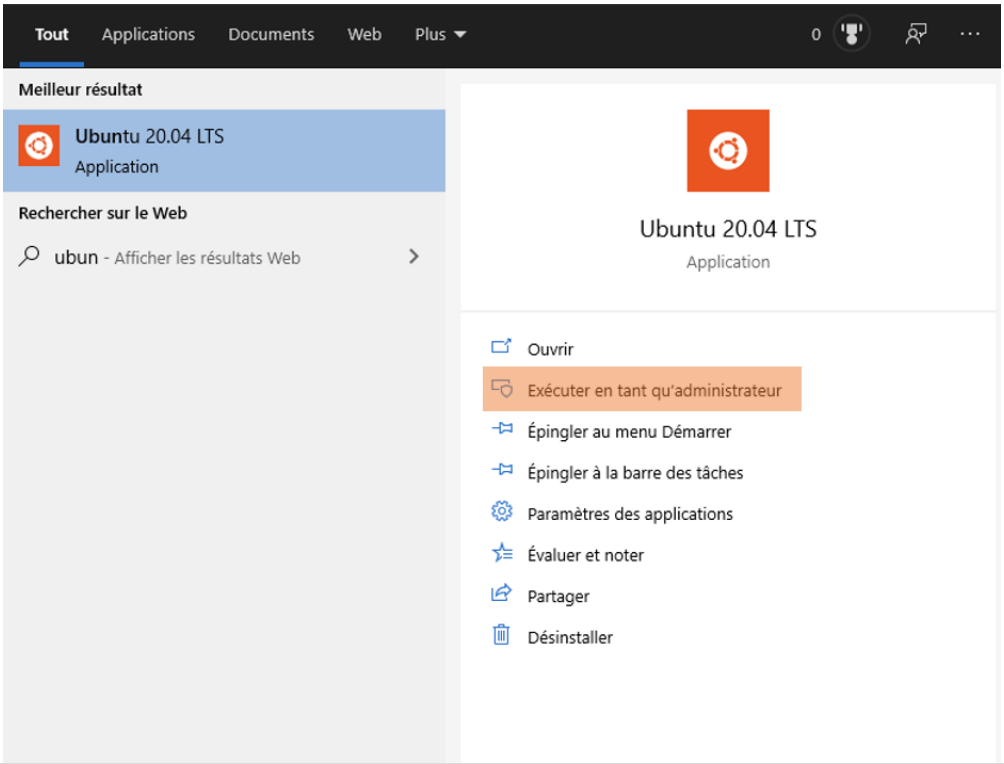
8) Cliquer sur télécharger



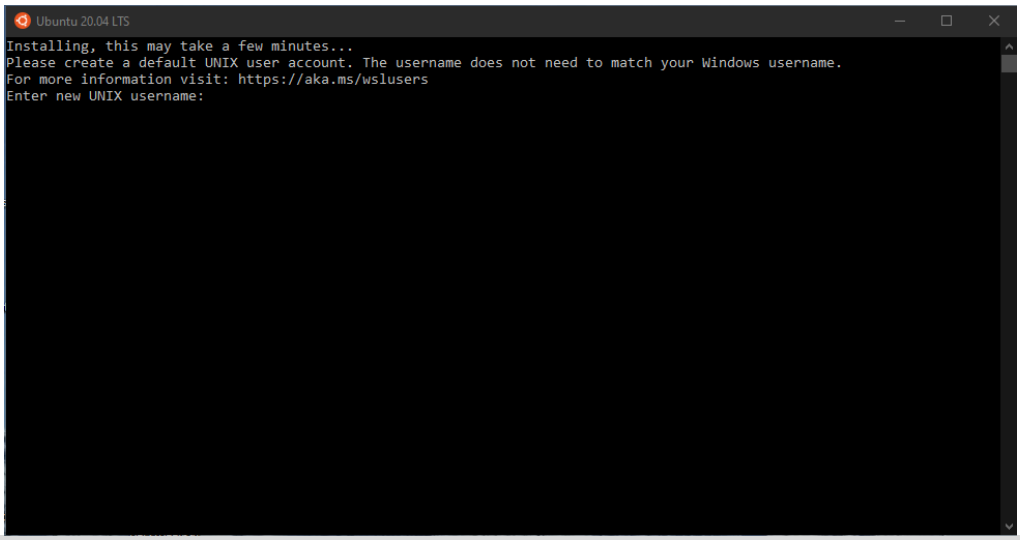
9) Cliquer sur installer



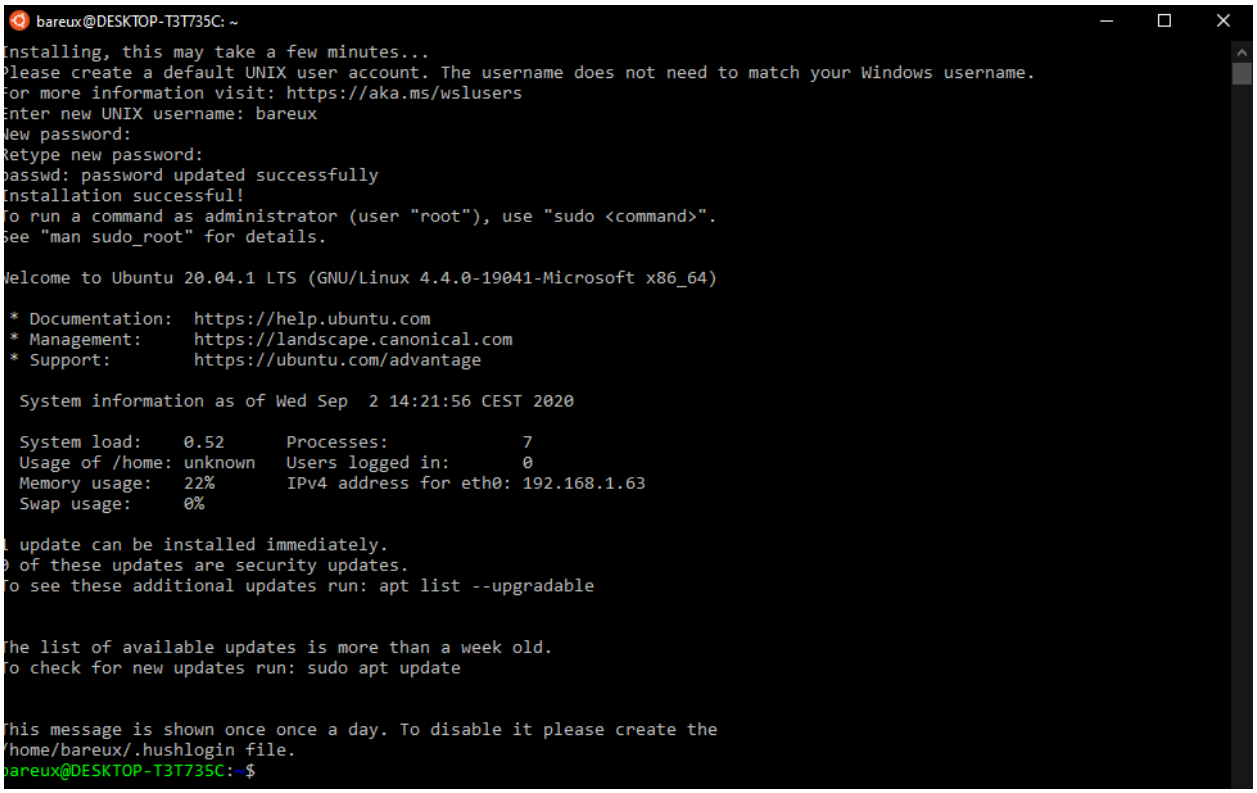
10) Dans la barre de recherche rechercher l'application Ubuntu 20.04 LTS et exécuter là en tant qu'administrateur



11) Donner un nom



12) Puis le mot de passe (il est normal que rien ne s'affiche lorsque vous tapez le mot de passe)

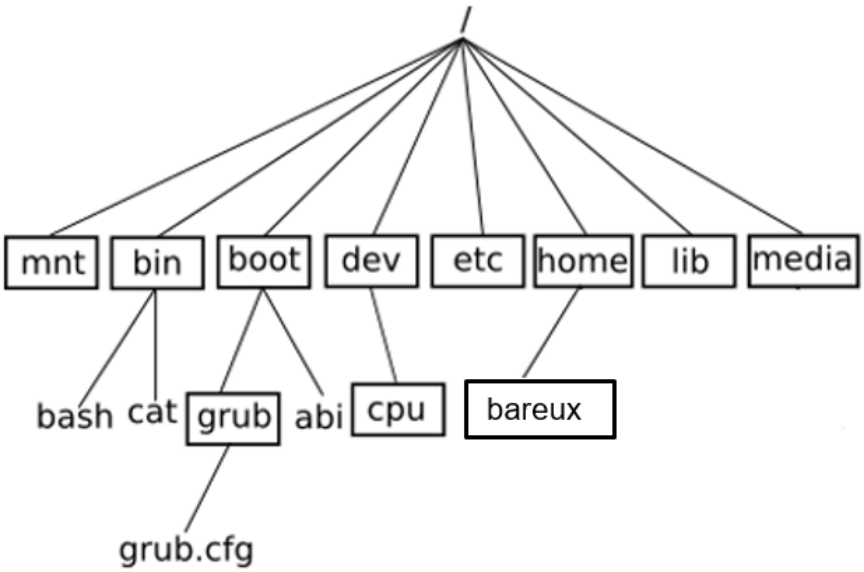


c) Les commandes de bases :

À début des systèmes d'exploitation, ces derniers étaient dépourvus d'interface graphique, toutes les interactions "système d'exploitation - utilisateur" se faisaient par l'intermédiaire de "lignes de commandes".

Nous voyons la console utilisateur « bareux » qui utilise l'ordinateur bareux@DESKTOP-T3T735C

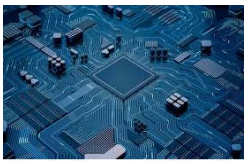
Dans la suite, nous allons travailler sur les fichiers et répertoires. Dans le système de type « Unix » , il y a un système de fichier en arborescence :



Dans le schéma ci-contre, on trouve des répertoires (noms entourés d'un rectangle, exemple : "home") et des fichiers (uniquement des noms "grub.cfg").
On parle d'arborescence, car ce système de fichier ressemble à un arbre à l'envers.

13) Taper la commande « ls / » et valider.

/bin, /sbin, /lib
Le répertoire **/bin** contient les exécutables (binaires) de base nécessaire au fonctionnement du système.
Toutes les commandes d'administration système essentielles sont, quant à elles, dans **/sbin** ("super" binaires).
Le répertoire **/lib** (libraries) regroupe les bibliothèques utilisées par les binaires contenus dans /bin et /sbin.



En comparaison avec un système Windows, ces trois répertoires correspondent aux exécutables contenus dans C:\windows
Ces trois répertoires sont vitaux pour le système et doivent être sur la même partition que /.

/boot:
Ce répertoire contient le noyau Linux et d'autres fichiers lancés à l'amorçage du système.
Ce répertoire peut faire l'objet d'une partition distincte

/dev: contient les fichiers périphériques
Une des particularités de Linux est de présenter, sous forme de fichiers, les périphériques attachés au système.
/dev/sda représentent le premier disque dur
/dev/cdrom représentent le lecteur de CD-Roms

/home: nommé également Home Directory = répertoires personnels
Ainsi pour les comptes utilisateur bareux, linus existeront les répertoires /home/bareux, /home/linus.
C'est dans son répertoire personnel qu'un utilisateur stocke ses fichiers de données et que viennent s'inscrire les fichiers de configuration propres à l'utilisateur.

/root: Home Directory du super-utilisateur.
Il ne se trouve pas dans /home pour des raisons de sécurité et parce qu'il ne peut faire l'objet d'une partition distincte comme /home.

/etc: contient les fichiers de configuration du système et des services. Répertoire important !

/tmp : espace d'échange temporaire, accessible a tous, effacé au démarrage.

/usr : contient les fichiers nécessaires aux applications (manuels, fichiers sources, librairies).

/var : contient les données des services et les journaux.

Fichiers cachés : sous Unix, les fichiers cachés commencent par un point. Par exemple, ~/.bashrc est un fichier caché, dans le répertoire personnel de l'utilisateur, qui contient la configuration de son shell.

14) Taper « cd / » puis « ls » que remarquez-vous ?

<u>Notion de chemin relatif et absolu</u>

Un chemin **absolu** commence toujours par la racine (/) → cd home/bareux/Documents

Un chemin **relatif** est un chemin qui dépend du dossier dans lequel vous vous trouvez : si vous êtes dans votre home directory /home/bareux/ → cd Documents

15) Donner et tester la commande pour retourner dans votre répertoire dans home puis faites un « ls »?

En général, les commandes ont une syntaxe de type: nom_commande [options] [paramètres]
Le nom de la commande est obligatoire. Les options et paramètres sont optionnels.

```
ls [OPTION]... [FILE]...
```

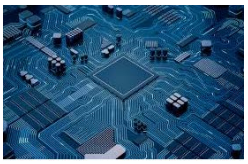
cd <chemin>: change de répertoire

Caractère **~** : désigne le répertoire HomeDirectory de l'utilisateur courant
Caractère **.** : désigne le répertoire courant
Caractère **..** : désigne le répertoire parent
Caractère **-** : désigne le répertoire précédent
Caractère **/** : désigne le répertoire racine

16) A la place de la commande précédente (Q26) pour retourner dans home, qu'aurait-on pu taper en plus court?

17) En vous basant sur l'arborescence ci-dessus, déterminez le chemin absolu permettant d'accéder au fichier « cat » ?

ls <chemin>: liste le contenu d'un répertoire



- Option **-a** : affiche les fichiers cachés
- Option **-l** : affichage long
- Option **-h** : human readable (lisible par un humain)
- Option **-t** : trier par date de dernière modification
- Option **-t -r** : renverse l'ordre
- Option **-larth** : le tout combiné

mkdir <chemin>: création d'un répertoire

rmdir <chemin>: suppression d'un répertoire

touch <fichier>: actualisation des dates d'un fichier ou création d'un fichier vide

rm <fichier>: suppression d'un fichier

cp <source...> <destination>: copie d'un fichier.

pwd : affiche le chemin absolu du répertoire dans lequel vous vous trouvez (**p** rint **w** ork **d** irectory)

```
bareux@DESKTOP-T3T735C:~$ pwd
/home/bareux
```

mv <source...> <destination>: déplacement ou renommage

rm : suppression
- Option **-r** : de manière récursive : dossier + sous dossiers
- Option **-i** : de manière interactive : demande confirmation
- Option **-f** : forcer la suppression

Pour chaque commande, il existe une page de manuel qui explique son fonctionnement.
Syntaxe: **man** <commande>

sudo : Super User DO permet de mettre l'utilisateur en super administrateur (en général nécessaire pour installer des modules supplémentaires).

Le chainage de commande est possible avec **" ; "** Exemple : **« cd / ; ls »**

En plus du chainage classique, les commandes peuvent être **liées et exécutées de façon conditionnelle**
La condition d'exécution d'une commande est la **réussite ou non de la précédente**.
Les caractères **&&** et **||** permettent d'effectuer une exécution conditionnelle.
Exemple **: sudo apt update && sudo apt upgrade**

18) Donner la commande et créer le répertoire « test_nsi »dans votre répertoire personnel ?

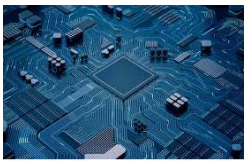
19) Donner la commande et créer un fichier « fic.txt » dans le répertoire « test_nsi » ?

20) Donner la commande et créer le répertoire «copie » puis copier le fichier « fic.txt » de « test_nsi » dans « copie »?

21) Donner la commande et supprimer le répertoire « copie » ?

d) Gestion des utilisateurs et des groupes :

Les systèmes de type "UNIX" sont des systèmes multi-utilisateurs, plusieurs utilisateurs peuvent donc partager un même ordinateur, chaque utilisateur possédant un environnement de travail qui lui est propre.
Chaque utilisateur possède certains droits lui permettant d'effectuer certaines opérations et pas d'autres.
Le système d'exploitation permet de gérer ces droits très finement. Un utilisateur un peu particulier possède tous les droits : ce "super utilisateur" est appelé "administrateur" ou "root".
C'est cet "administrateur" qui pourra attribuer ou retirer certains droits aux autres utilisateurs. Au lieu de gérer les utilisateurs un par un, il est possible de créer des groupes d'utilisateurs. L'administrateur attribue des droits à un groupe au lieu d'attribuer des droits particuliers à chaque utilisateur.
Nous nous intéresserons ici uniquement aux droits liés aux fichiers, mais il existe d'autres droits liés aux autres éléments du système d'exploitation ((imprimante, installation de logiciels...).



Les fichiers et les répertoires possèdent 3 types de droits :
- les droits en **lecture** (symbolisés par la lettre **r**) : est-il possible de lire le contenu de ce fichier
- les droits en **écriture** (symbolisés par la lettre **w**) : est-il possible de modifier le contenu de ce fichier
- les droits en **exécution** (symbolisés par la lettre **x**) : est-il possible d'exécuter le contenu de ce fichier.

Il existe 3 types d'utilisateurs pour un fichier ou un répertoire :
- le **propriétaire du fichier** (personne qui a créé le fichier), il est symbolisé par la lettre **u**
- un **fichier est associé à un groupe**, tous les utilisateurs appartenant à ce groupe possèdent des droits particuliers sur ce fichier. Le groupe est symbolisé par la lettre **g**
- **tous les autres utilisateurs** (ceux qui ne sont pas le propriétaire du fichier et qui n'appartiennent pas au groupe associé au fichier). Ces utilisateurs sont symbolisés la lettre **o**

22) Dans le répertoire « test_nsi », taper la commande « ls -l », interprétez ce que vous obtenez ?

```
bareux@DESKTOP-T3T735C:~/test_nsi$ ls -l
total 0
-rwxr-xr-- 1 bareux bareux 0 Sep  2 14:44 fic.txt
```

Lisons cette ligne de gauche à droite :
- le premier symbole "-" signifie que l'on a affaire à un fichier, dans le cas d'un répertoire, nous aurions un "d"
- les 3 symboles suivants "rw-"donnent les droits du propriétaire du fichier : lecture autorisée (r), écriture autorisée (w), exécution interdite (- à la place de x)
- les 3 symboles suivants "r--"donnent les droits du groupe lié au fichier : lecture autorisée (r), écriture interdite (- à la place de w), exécution interdite (- à la place de x)
- les 3 symboles suivants "r--"donnent les droits des autres utilisateurs : lecture autorisée (r), écriture interdite (- à la place de w), exécution interdite (- à la place de x)
- le caractère suivant "1" donne le nombre de liens (nous n'étudierons pas cette notion ici)
- le premier "bareux" représente le nom du propriétaire du fichier
- le second "bareux" représente le nom du groupe lié au fichier
- le "0" représente la taille du fichier en octet (ici notre fichier est vide)
- "sept. 8_11 :03 " donne la date et l'heure de la dernière modification du fichier
- "fic.txt" est le nom du fichier

Gestion des droits :

La gestion des droits s'effectue avec les 3 commandes suivantes :
Chown : permet de changer l'utilisateur propriétaire d'un fichier /répertoire
Chgrp : permet de changer le groupe propriétaire d'un fichier / répertoire
Chmod : permet de modifier les autorisations d'un fichier / répertoire
L'option **-R** indique à la commande d'appliquer la modification de façon récursive.

Il y a deux manières d'indiquer les droits d'accès d'un fichier sous Unix : en notation **symbolique** et en notation **octale**.

Symbolique :

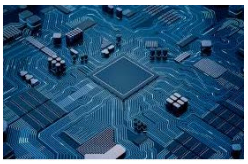
La notation symbolique s'appuie sur les caractères :
r (lecture) **w** (écriture) **x** (exécution) pour désigner les droits et **u** (utilisateur) **g** (groupe) **o** (autres) pour symboliser les entités concernées.

Dans la notation symbolique, le caractère **a** est équivalent à **ugo** et représente les trois entités.

Un **+** dans cette syntaxe indique qu'il faut positionner le droit, un **-** indique que ce droit doit être supprimé et un **=** permet de définir les droits en une seule étape pour l'entité spécifiée.

La syntaxe générale ressemble alors à : <entité(s)>[+-=]<droit(s)>
Par exemple :

Commande	Résultat
u+x	Ajoute (+) le droit d'exécution (x) pour le propriétaire (u).
g-w	Supprime (-) le droit de modification (w) pour le groupe (g).
o-rw	Supprime (-) les droits de lecture (r) et de modification (w) pour les autres (o).
a+r	Ajoute (+) le droit de lecture (r) pour toutes les entités (a).
u=rw	Fixe (=) les droits de lecture (r) et de modification (w) et supprime le droit d'exécution (x) pour le propriétaire (u).

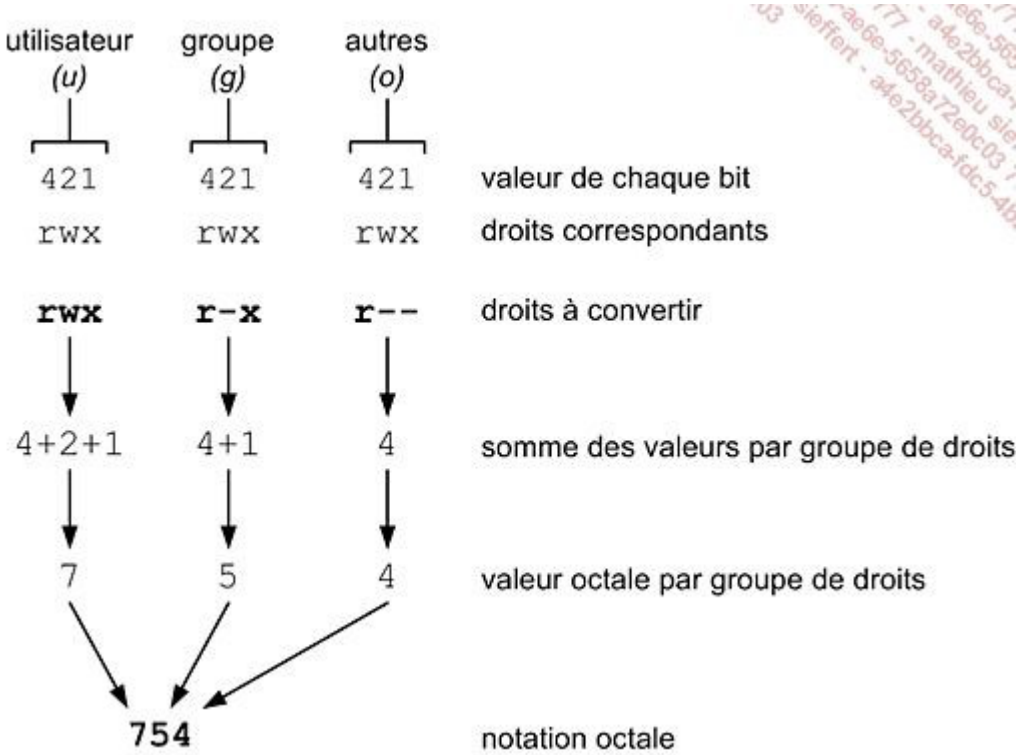


Octale :

Cette seconde notation est le codage sur 3 bits des droits de chaque entité.
En positionnant le bit à 1 lorsque le droit correspondant est autorisé, et à 0 lorsqu'il n'est pas permis, cela donne par exemple 101 pour les droits r-x.

Droits (symbolique)	Notation binaire	Notation octale
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

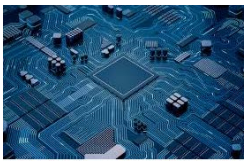
Par exemple :



23) Faire chmod 754 « fic.txt » puis « ls -l », interprétez ce que vous obtenez ?

24) A quels droits correspondent les entiers :
- 751 :
- 521 :
- 214 :
- 150 :

25) Par quels entiers sont codés les droits :
- r w - r- - r - - :
- r w x r – x r – x :



Mise en place d'un espace de travail pour 4 utilisateurs

https://wiki.debian-fr.xyz/Commandes_utilisateurs_et_groupes

Pour ajouter un utilisateur à l'aide de la commande adduser: **adduser <utilisateur> en option <groupe>**

Pour définir un mot de passe pour un utilisateur : **passwd <utilisateur>**

Pour supprimer un utilisateur : **userdel -r <utilisateur>**

Pour ajouter un groupe à l'aide de la commande addgroup **: addgroup -- force-badname <groupe>**

Pour changer les groupes propriétaire pour ces dossiers : **chgpr <groupe> <répetroire>**

26) Créer 4 utilisateurs : user1, user2, user3 et user 4

27) Créer 2 groupes groupeA et groupeB

28) Ajouter les utilisateurs user1 et user2 dans le groupeA

29) Ajouter les utilisateurs user3 et user4 dans le groupeB

Pour visualiser les membres d'un groupe : cat /etc/group

30) Créer un dossier /home/groupeA et un dossier /home/groupeB

31) Changer le groupe propriétaire pour ces dossiers (respectivement groupeA et groupeB)

32) Modifier les droits pour que seuls les membres du groupe aient un accès en lecture / écriture sur le dossier du groupe

33) Lister les droits de ces 2 répertoires